

picoDAW: a web-based sequencer/synthesizer/sampler

Tim Fan (SID: 550790019)

hourlilies

<https://github.com/hourlilies/elec5305-project-550790019>

I. PROJECT OVERVIEW

This project aims to develop a music sequencer with a synthesizer and sampler as available instruments, for any device capable of running a modern web browser supporting WebAudio, using the emscripten compiler toolchain that turns C++ DSP code into WebAssembly.

The goal is to make music production more accessible; pretty much any computer—whether that be a smartphone, laptop, or even a single-board computer—can run a modern web browser that supports WebAudio. This is important because anyone should be able to play with and make music, without needing to install and learn a comprehensive desktop-based music production application designed for professionals and not the general public.

Though the advent of generative AI has made music creation much more accessible, it has come at the cost of abstracting away the underlying signal processing and audio intuitions that makes it all work. This project aims to expose all of that, the wires, knobs, and switches of music making. To poke at the machine that makes modern music tick, whose human invention and decades of production is what allows powerful GenAI tools like Suno or Udio to even exist.

II. DEFINITIONS

Before moving on, I want to briefly describe what I mean by a sequencer/synthesizer/sampler, and also what a digital audio workstation—aka DAW—is.

A. Synthesizer

What I mean by a synthesizer instrument, is something that generates and plays a sound—creates audio-frequency air pressure waves—on demand. So, when you press a key on the synthesizer’s input controller, the synthesizer will generate—aka synthesize—and play a sound, corresponding to what key was pressed. For example, a basic sawtooth synthesizer is a synthesizer that creates the sawtooth wave sound on demand, its frequency—aka pitch—depending on which key is played on the keyboard connected to it. Pressing the key corresponding to the pitch A5, will result in the synthesizer synthesizing and playing a sawtooth wave sound at 880Hz.

B. Sampler

A sampler instrument, in my understanding, is actually very similar, except that instead of generating a new sound, it will

take an existing sound—e.g. the wave format recording of a real-life snare drum being hit—and play it back on demand; possibly also modifying or processing it before playing it back. For example, pressing one key might play back the snare drum recording in its original form, literally re-playing the original wave recording. While pressing another key might play back the recording at half its sample rate, resulting in the same sound but lowered one octave.

C. Sequencer

At the core of both these instruments are their respective input controllers; typically used by the human body, whether that be through hitting it, blowing air into it, stepping on it, etc. The sequencer then, in my understanding, gets its name from the idea from an instrument that can be played, not by hitting it directly, but by providing it a sequence of instructions on what keys the instrument should press for itself, and when and in what order.

An example of such an instrument is the player piano¹, which reads the desired keys to be pressed from a sheet of paper, written by a human who punches holes in the paper corresponding to the keys they want the instrument to press, and when they should be pressed. In essence, like how early computers were programmed, through punch cards. Presumably, because the sheets of player piano paper could get very long, they would be rolled up, giving them their name; piano rolls².

A sequencer then, in my understanding, is the machine a human would use to write the program—e.g. a piano roll—that will then be loaded by an instrument that can understand this program; in this case, a player piano. Perhaps the most common type of sequencer, fittingly, is based off the piano roll. In REAPER³ for example, the “piano roll” mode of its sequencer is the default mode (fig. 1a). Other modes include the “musical notation” mode, which is based off the Western Art or classical music tradition of sheet music (fig. 1b).

D. Digital audio workstation

A typical DAW then, is an application that combines all of the above—sequencer/synthesizer/sampler—alongside many other features useful for professionals and experienced hobbyists.

¹https://en.wikipedia.org/wiki/Player_piano

²https://en.wikipedia.org/wiki/Piano_roll

³<https://www.reaper.fm>

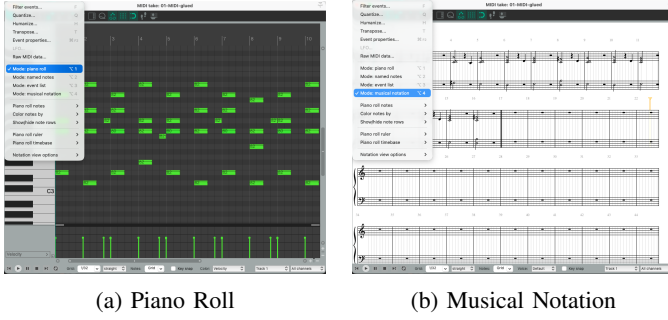


Fig. 1: The modes of REAPER's MIDI sequencer

For example, a DAW like the aforementioned REAPER will also allow you to record audio through it; the recording can then of course be played back through its sampler. Note that because the sampler is also an instrument, the timing of playback can also be sequenced through the included sequencer. And, though it comes with pre-installed instruments, third-party instruments can also be used with its sequencer, through integration with software instrument standards like the VST⁴.

III. BACKGROUND AND MOTIVATION

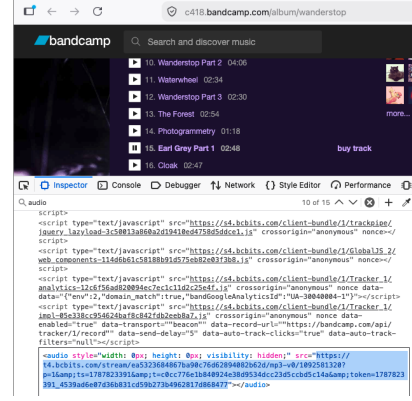
A. WebAudio

Despite its relatively recent history, with the first publication of a working draft in only 2011⁵, the WebAudio API has seen extensive development over the course of a decade, eventually receiving an endorsement from the W3C in 2021 as a Recommendation⁶. Notably, its primary `AudioContext` interface has been fully supported by most major web browsers since at least 2015⁷, with the only exception being Safari which implemented support in 2021.

Of course, audio has existed for a much longer time than that on the internet. As Boris Smus describes in their brief history of audio on the web [1], initially that was through the Flash external plugin for browsers, and then later through the HTML5 `<audio>` element which was instead natively supported by the web browser itself. In fact, it's this `<audio>` element that allows music streaming websites like Bandcamp to preview an artist's music through the web browser (fig. 2).

But one thing Flash's plugin did for web audio that HTML5's `<audio>` element doesn't do, is allow for interactive audio. For example, a synthesizer/sampler/sequencer like the one this project will aim to build, would require a predictable amount of latency between a person pressing a key on their keyboard and the corresponding sound being played through their speakers, something that the `<audio>` element just wasn't designed for.

That is the kind of problem the WebAudio API attempts to solve, and why it has been possible since then for synthesizers and other real time instruments to be built for the web. For



Input ... from '1092581320.mp3' Duration: 00:02:48.00, ..., bitrate: 236 kb/s

Fig. 2: Bandcamp allows artists to offer their listeners a compressed and lower bitrate MP3 preview of their tracks, played in the browser through the HTML5 `<audio>` element

example Steven Goldberg's emulation of the Juno-106 synthesizer⁸ built for the web in 2015, or Takamitsu Endo's collection of synthesizers⁹ built for the web starting in the 2020's, both using the aforementioned `AudioContext` interface as the backbone of their audio framework (for example, fig. 3); and many more examples, like all the web synthesizers available at playtronica¹⁰.

B. Web-based DAWs

There are also examples of projects for the web that combine a synthesizer with an integrated sequencer, in the same vein as what this project aims to do. One notable example is certainly André Michelle's `opendaw`¹¹, containing a sequencer, synthesizers, samplers, and more; everything you would expect from a conventional desktop-based DAW! However, even its interface strongly resembles a conventional desktop-based DAW, also requiring a desktop browser.

That is why, through this project, the author argues the need for a stripped-down DAW—consisting of just a sequencer/synthesizer/sampler; hence the name `picoDAW`—that is accessible (i) because it is built for the web, and (ii) has a simplified user interface, approachable not because it is the conventional interface that every other DAW uses, but because it is approachable.

IV. PROPOSED METHODOLOGY

proposal

V. EXPECTED OUTCOMES

outcomes.tex

VI. TIMELINE

timeline.tex

⁴https://en.wikipedia.org/wiki/Virtual_Studio_Technology

⁵<https://www.w3.org/TR/2011/WD-webaudio-20111215/>

⁶<https://www.w3.org/TR/webaudio-1.0/>

⁷https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API#browser_compatibility

⁸<https://github.com/stevengoldberg/juno106>

⁹<https://github.com/ryukau/UhhyouWebSynthesizers>

¹⁰<https://synth.playtronica.com/>

¹¹<https://opendaw.studio>

```

// Copyright Takamitsu Endo (ryukau@gmail.com)
// SPDX-License-Identifier: Apache-2.0

export class Audio {
  cachedBuffer = null;
  constructor() {
    // AudioContext is the backbone of WebAudio
    this.audioContext = new AudioContext();

    // Custom class that represents .wav data
    this.wave = new Wave();
  }
  play() {
    if (!this.cachedBuffer) {
      // Create and write .wav data to buffer
      let buffer = this.audioContext.createBuffer(
        this.audioContext.sampleRate);
      for (let i = 0; i < this.wave.channels; ++i) {
        buffer.copyToChannel(new Float32Array(
          this.wave.data[i]), i, 0);
      }
      this.cachedBuffer = buffer;
    }

    // Set AudioContext's buffer to our one
    this.source =
      this.audioContext.createBufferSource();
    this.source.buffer = cachedBuffer;

    // Play the .wav data through AudioContext
    this.source.start();
  }
}

```

Fig. 3: Heavily redacted version of the custom Audio class used by ryukau⁹ for their web synthesizer plugins, built on the WebAudio API and located at `common/wave.js`

REFERENCES

- [1] B. Smus, *Web audio API: advanced sound for games and interactive apps*. O'Reilly Media, Inc., 2013.